Reviewer Pack — Minimal Hodge Diamond Diameter and Communication Complexity …

Entry a007c825ff6c · INCONCLUSIVE

SEC autonomous research engine — attribution: Ludovico Kubler

2026-06-05 01:21:59 UTC

Minimal Hodge Diamond Diameter and Communication Complexity Rank

Entry ID: `a007c825ff6c` **Verdict**: `INCONCLUSIVE` **Recorded**: 2026-06-05 01:21:59 UTC

1. Conjecture

Field A (mathematical branch): Algebraic Geometry (Hodge Theory) **Field B** (complexity object): Communication Complexity

Statement:

> For any given boolean function f with communication complexity rank r(f), the diameter of the minimal Hodge diamond for its associated algebraic variety is bounded by O(r(f)).

Rationale (proposer's reasoning):

> The Hodge diamond provides a geometric representation of the cohomology groups of an algebraic variety, which could capture structural information about the boolean function's complexity. The conjecture suggests that this geometric structure is related to communication complexity, potentially revealing new insights into computational hardness.

Taxonomy category: `algebraic_geometry_communication_complexity` (status at proposal time:)

2. Pre-registration (Popper-style)

Hash (SHA-256 prefix): `f284ac98736b7ab6`

This hash commits to the conjecture statement, acceptance criterion, and seed list **before** the empirical test runs. Tampering with the test or its data after this hash was computed would be detectable.

Acceptance criterion:

> The conjecture is supported if the mean Hodge diamond diameter across all 30 seeds is less than or equal to three times the median communication complexity rank, and no seed's Hodge diamond diameter exceeds the median communication complexity rank by more than 3.

3. Barrier filter (F1)

Final: `PASS`

Barrier	Verdict	Confidence	LLM₁	LLM₂
RELATIVIZATION	SAFE	0.50	UNCERTAIN	UNCERTAIN
NATURAL_PROOFS	SAFE	0.50	UNCERTAIN	UNCERTAIN
ALGEBRIZATION	SAFE	0.50	UNCERTAIN	UNCERTAIN
KARP_LIPTON	SAFE	1.00	UNCERTAIN	SAFE

4. Novelty audit

Verdict: NOVEL against 0 hits across arXiv + Semantic Scholar + ECCC

Search queries (3): - algebraic geometry AND hodge theory AND communication complexity - minimal hodge diamond diameter AND communication complexity rank - boolean function AND communication complexity rank AND hodge theory

5. Empirical test harness

Multi-seed protocol: 5 seeds (11, 23, 37, 53, 71)
Sandbox: isolated subprocess, stdlib-only, ≤90s wall time per cycle **Execution**: rc=0, elapsed=0.1s

5.1 Generated Python source

```python
import random
import math

def run_trial(seed: int) -> dict:
    random.seed(seed)

    # Define the communication complexity rank r(f)
    n = 10  # Example size, can be adjusted
    r_f = n  # Simple example of communication complexity rank

    # Construct the associated algebraic variety (simplified for demonstration)
    # This is a placeholder and should be replaced with actual computation
    hodge_diamond = [[i * j for j in range(n)] for i in range(n)]

    # Calculate the Hodge diamond diameter
    max_distance = 0
    for i in range(n):
        for j in range(n):
            distance = abs(i - j)
            if distance > max_distance:
                max_distance = distance

    metric_name = "Hodge Diamond Diameter"
    metric_value = max_distance
    instances_tested = 1
    n_max = n
    conjecture_holds = False
    counterexample = "mapping_undefined"

    return {
        "metric_name": metric_name,
        "metric_value": metric_value,
        "instances_tested": instances_tested,
```

```
        "n_max": n_max,
        "conjecture_holds": conjecture_holds,
        "counterexample": counterexample
    }

if __name__ == "__main__":
    seeds = [int(x) for x in sys.argv[1:]] or [2, 3, 5, 7, 11, 13, 17, 19, 23, 29] * 3
    results = []

    for seed in seeds:
        result = run_trial(seed)
        print(f"TRIAL: {result}")
        results.append(result)

    mean_metric_value = sum(r["metric_value"] for r in results) / len(results)
    std_metric_value = math.sqrt(sum((r["metric_value"] - mean_metric_value) ** 2 for r in results
    support_fraction = sum(1 for r in results if r["conjecture_holds"]) / len(results)

    if all(r["conjecture_holds"] for r in results):
        print(f"RESULT: SUPPORTED mean={mean_metric_value} std={std_metric_value} support_fraction
    elif any(not r["conjecture_holds"] for r in results) and support_fraction >= 0.8:
        first_failing_seed = next(r["seed"] for r in results if not r["conjecture_holds"])
        print(f"RESULT: FALSIFIED counterexample='mapping_undefined' first_failing_seed={first_fai
    else:
        print("RESULT: INCONCLUSIVE insufficient_data")
```

6. Per-seed results

(no seed-level data — test crashed before producing TRIAL: lines)

7. Test stdout (last 2KB)

```
tested': 1, 'n_max': 10, 'conjecture_holds': False, 'counterexample': 'mapping_undefined'}
TRIAL: {'metric_name': 'Hodge Diamond Diameter', 'metric_value': 9, 'instances_tested': 1, 'n_max': 1
TRIAL: {'metric_name': 'Hodge Diamond Diameter', 'metric_value': 9, 'instances_tested': 1, 'n_max': 1
TRIAL: {'metric_name': 'Hodge Diamond Diameter', 'metric_value': 9, 'instances_tested': 1, 'n_max': 1
TRIAL: {'metric_name': 'Hodge Diamond Diameter', 'metric_value': 9, 'instances_tested': 1, 'n_max': 1
TRIAL: {'metric_name': 'Hodge Diamond Diameter', 'metric_value': 9, 'instances_tested': 1, 'n_max': 1
TRIAL: {'metric_name': 'Hodge Diamond Diameter', 'metric_value': 9, 'instances_tested': 1, 'n_max': 1
TRIAL: {'metric_name': 'Hodge Diamond Diameter', 'metric_value': 9, 'instances_tested': 1, 'n_max': 1
TRIAL: {'metric_name': 'Hodge Diamond Diameter', 'metric_value': 9, 'instances_tested': 1, 'n_max': 1
TRIAL: {'metric_name': 'Hodge Diamond Diameter', 'metric_value': 9, 'instances_tested': 1, 'n_max': 1
TRIAL: {'metric_name': 'Hodge Diamond Diameter', 'metric_value': 9, 'instances_tested': 1, 'n_max': 1
TRIAL: {'metric_name': 'Hodge Diamond Diameter', 'metric_value': 9, 'instances_tested': 1, 'n_max': 1
RESULT: INCONCLUSIVE insufficient_data
```

8. Critic adversarial review

Critic verdict: `CHALLENGE`

Critic reasoning:

> The test code uses a fixed size n = 10 for all trials, which is too small to provide meaningful evidence for the conjecture. The communication complexity rank r(f) and the associated algebraic variety are not properly computed or defined in the code.

9. Final verdict & safety rail

Verdict: `INCONCLUSIVE`

Reasoning:

> Safety rail: critic_challenge_falsified | original: The test results indicate that the conjecture does not hold for at least one instance (counterexample provided), and the critic challenges the methodo | next: Re-evaluate the conjecture with a larger sample size and ensure that communication complexity rank and associated algebraic variety are correctly computed. Additionally, consider using a more robust statistical method to validate the conjecture.

11. Audit log (LLM calls)

Total LLM calls: 12

#	Phase	Provider	Model	Tokens in	out	Latency (ms)
1	propose	ollama_remote	glm4:latest	0	0	13392
2	propose	ollama_remote	glm4:latest	0	0	14725
3	propose	ollama_remote	glm4:latest	0	0	11861
4	preregistration	ollama_remote	glm4:latest	0	0	9534
5	novelty	ollama_remote	glm4:latest	0	0	11585
6	novelty	ollama_remote	glm4:latest	0	0	11975
7	test_gen	ollama_remote	qwen2.5-coder:7b	0	0	13438
8	test_gen	ollama_remote	qwen2.5-coder:7b	0	0	11076
9	test_gen	ollama_remote	qwen2.5-coder:7b	0	0	15617
10	test_gen	ollama_remote	qwen2.5-coder:7b	0	0	7342
11	critic	ollama_remote	glm4:latest	0	0	18809
12	judge	ollama_remote	glm4:latest	0	0	9677

Totals: 0 input tokens, 0 output tokens, ~$0.0000 cost, 149030 ms total latency. Provider mix: {'ollama_remote': 12}

(full prompt+response transcripts available in `research/audit/a007c825ff6c.jsonl`)

12. Reproducibility

To reproduce this cycle:

1. Apply the test harness in section 5.1 with seeds 11,23,37,53,71
2. The Python sandbox is pure-stdlib; any Python 3.11+ should suffice
3. The full LLM transcripts are in `research/audit/a007c825ff6c.jsonl` for verification of provenance
4. The pre-registration hash in section 2 commits to the test design before execution; tampering would be detectable.

Sandbox file: `research/pvsnp_sandbox/test_*.py` (per-cycle)
Replay tarball: `research/replay/a007c825ff6c.tar.gz` (if generated)